

Major types of Indexes in SQL Server, with real-time scenario-based examples using AdventureWorks 2019.

Learnings: concept → syntax → scenario → demo query for each type.

Indexes in SQL Server – With AdventureWorks 2019 Examples

Indexes are like “roadmaps” to find data quickly. The wrong index = poor performance, the right index = query boosts in milliseconds.

1. Clustered Index

- **Definition:** Determines the physical order of rows in the table. Only **one clustered index** per table.
- **Default:** If you create a Primary Key, SQL Server creates a clustered index by default (unless specified otherwise).

Scenario:

The **Sales.SalesOrderHeader** table has millions of rows. Queries often filter on SalesOrderID.

-- Create clustered index

```
CREATE CLUSTERED INDEX IX_SalesOrderHeader_SalesOrderID
```

```
ON Sales.SalesOrderHeader (SalesOrderID);
```

-- Query benefiting

```
SELECT SalesOrderID, OrderDate, TotalDue
```

```
FROM Sales.SalesOrderHeader
```

```
WHERE SalesOrderID = 50000;
```

👉 Rows are physically stored by SalesOrderID, so lookups are super-fast.

2. Non-Clustered Index

- **Definition:** A separate structure that stores indexed columns + a pointer (RID/Clustered Key) to actual data.
- **Multiple allowed** per table.

Scenario:

Person.Person table has frequent searches on LastName.

-- Non-clustered index

```
CREATE NONCLUSTERED INDEX IX_Person_LastName
```

```
ON Person.Person (LastName);
```

-- Query benefiting

```
SELECT BusinessEntityID, FirstName, LastName
```

```
FROM Person.Person
```

```
WHERE LastName = 'Smith';
```

👉 Speeds up lookups without scanning the entire table.

3. Unique Index

- **Definition:** Ensures uniqueness of values in a column.
- **Automatically created** when defining a UNIQUE constraint.

Scenario:

In Person.EmailAddress, email IDs must be unique.

```
CREATE UNIQUE INDEX IX_Person_Email
```

```
ON Person.EmailAddress (EmailAddress);
```

👉 Prevents duplicates + speeds lookups.

4. Filtered Index

- **Definition:** Non-clustered index with a **WHERE clause** → smaller, more efficient.

Scenario:

In Sales.SalesOrderHeader, many orders are Completed, but queries often fetch only Pending ones.

```
CREATE NONCLUSTERED INDEX IX_SalesOrder_PendingOrders  
ON Sales.SalesOrderHeader (OrderDate, CustomerID)  
WHERE Status = 1; -- 1 = Pending
```

👉 Saves space, improves performance for selective queries.

5. Composite Index (Multi-Column)

- **Definition:** Index on multiple columns.

Scenario:

Frequent search on LastName + FirstName.

```
CREATE NONCLUSTERED INDEX IX_Person_Name  
ON Person.Person (LastName, FirstName);
```

-- Query benefiting

```
SELECT BusinessEntityID, LastName, FirstName  
FROM Person.Person  
WHERE LastName = 'Johnson' AND FirstName = 'Emily';
```

👉 Best when queries filter/join on multiple columns together.

6. Covering Index (with INCLUDE)

- **Definition:** Index includes extra non-key columns to **avoid key lookups**.

Scenario:

Query on SalesOrderHeader retrieves OrderDate and TotalDue, but index is only on CustomerID.

```
CREATE NONCLUSTERED INDEX IX_SalesOrder_Customer  
ON Sales.SalesOrderHeader (CustomerID)  
INCLUDE (OrderDate, TotalDue);
```

-- Query benefiting

```
SELECT CustomerID, OrderDate, TotalDue  
FROM Sales.SalesOrderHeader  
WHERE CustomerID = 11000;
```

👉 Query is fully satisfied by index, no base table lookup.

7. Columnstore Index

- **Definition:** Stores data column-wise (not row-wise). Great for analytics/warehousing.
- **Types:** Clustered Columnstore (CCI), Non-Clustered Columnstore (NCCI).

Scenario:

FactInternetSales table (DW style reporting, heavy aggregations).

-- Clustered Columnstore Index

```
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales  
ON dbo.FactInternetSales;
```

-- Query benefiting

```
SELECT ProductKey, SUM(SalesAmount) AS TotalSales
FROM dbo.FactInternetSales
GROUP BY ProductKey;
```

👉 Aggregations run **10x faster** due to compression + batch processing.

8. XML Index

- **Definition:** Optimizes queries on XML columns (Primary & Secondary XML Indexes).

Scenario:

Production.ProductModel has CatalogDescription XML column.

-- Primary XML Index

```
CREATE PRIMARY XML INDEX PXML_ProductModel_Catalog
ON Production.ProductModel (CatalogDescription);
```

-- Query benefiting

```
SELECT Name
FROM Production.ProductModel
WHERE CatalogDescription.value('(/ProductDescription/Summary)[1]', 'nvarchar(100)') LIKE '%bike%';
```

👉 Speeds up XQuery/XML value retrieval.

9. Spatial Index

- **Definition:** Indexes spatial data types (geometry, geography).

Scenario:

Suppose a table dbo.StoreLocations with GeoLocation column.

-- Create spatial index

```
CREATE SPATIAL INDEX SInd_StoreLocations
ON dbo.StoreLocations (GeoLocation);
```

-- Query benefiting

```
SELECT StoreName
FROM dbo.StoreLocations
WHERE GeoLocation.STDistance(geography::Point(40.7128, -74.0060, 4326)) < 5000;
```

👉 Helps fast geo-spatial proximity searches (e.g., stores near New York).

10. Full-Text Index

- **Definition:** Allows rich text searches (words, phrases, linguistic analysis).

Scenario:

Production.Document has large text data in DocumentSummary.

-- Enable full-text

```
CREATE FULLTEXT CATALOG ftCatalog AS DEFAULT;
```

```
CREATE FULLTEXT INDEX ON Production.Document
(DocumentSummary LANGUAGE 1033)
KEY INDEX PK_Document_DocumentID;
```

-- Query benefiting

SELECT DocumentID, Title

FROM Production.Document

WHERE CONTAINS(DocumentSummary, ' "safety helmet" OR "protective gear" ');

👉 Ideal for searching documents, product descriptions, logs.

Summary Table

Index Type	Best For	AdventureWorks Example
Clustered	PK lookups	SalesOrderHeader(SalesOrderID)
Non-Clustered	Frequent search	Person.Person(LastName)
Unique	Enforcing uniqueness	Person.EmailAddress(EmailAddress)
Filtered	Subset queries	SalesOrderHeader(Status=1)
Composite	Multi-col filters	Person(LastName, FirstName)
Covering (INCLUDE)	Avoid lookups	SalesOrderHeader(CustomerID INCLUDE OrderDate, TotalDue)
Columnstore	Analytics/reporting	FactInternetSales
XML	XML queries	ProductModel(CatalogDescription)
Spatial	Geo queries	StoreLocations(GeoLocation)
Full-Text	Text search	Production.Document(DocumentSummary)

Step-by-step demo lab script for each index type using **AdventureWorks2019**.

This will help you show **before vs after** performance with **Execution Plans** in a training/webinar.

SQL Server Index Demo Lab (AdventureWorks2019)

 **Note:** Before each demo, run the query **without the index**, check **Execution Plan** & I/O stats, then create the index & rerun.

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

1 Clustered Index

Scenario: Optimize search by SalesOrderID.

-- Query BEFORE index

```
SELECT SalesOrderID, OrderDate, TotalDue  
FROM Sales.SalesOrderHeader  
WHERE SalesOrderID = 50000;
```

-- Create Clustered Index

```
CREATE CLUSTERED INDEX IX_SalesOrderHeader_SalesOrderID  
ON Sales.SalesOrderHeader (SalesOrderID);
```

-- Query AFTER index

```
SELECT SalesOrderID, OrderDate, TotalDue  
FROM Sales.SalesOrderHeader  
WHERE SalesOrderID = 50000;
```

 Execution Plan: Table Scan → Index Seek.

2 Non-Clustered Index

Scenario: Search customers by LastName.

-- Query BEFORE

```
SELECT BusinessEntityID, FirstName, LastName  
FROM Person.Person  
WHERE LastName = 'Smith';
```

-- Create Non-Clustered Index

```
CREATE NONCLUSTERED INDEX IX_Person_LastName  
ON Person.Person (LastName);
```

-- Query AFTER

```
SELECT BusinessEntityID, FirstName, LastName  
FROM Person.Person  
WHERE LastName = 'Smith';
```

3 Unique Index

Scenario: Enforce unique emails.

-- Create Unique Index

```
CREATE UNIQUE INDEX IX_Email_Unique
```

```
ON Person.EmailAddress (EmailAddress);
```

-- Test violation

```
INSERT INTO Person.EmailAddress (BusinessEntityID, EmailAddress, rowguid, ModifiedDate)
```

```
VALUES (1, 'ken0@adventure-works.com', NEWID(), GETDATE()); -- should fail
```

4 Filtered Index

Scenario: Optimize query for **Pending orders only**.

-- Query BEFORE

```
SELECT SalesOrderID, OrderDate, Status
```

```
FROM Sales.SalesOrderHeader
```

```
WHERE Status = 1;
```

-- Create Filtered Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrder_Pending
```

```
ON Sales.SalesOrderHeader (OrderDate, CustomerID)
```

```
WHERE Status = 1;
```

-- Query AFTER

```
SELECT SalesOrderID, OrderDate, Status
```

```
FROM Sales.SalesOrderHeader
```

```
WHERE Status = 1;
```

5 Composite Index

Scenario: Search by LastName + FirstName.

-- Query BEFORE

```
SELECT BusinessEntityID
```

```
FROM Person.Person
```

```
WHERE LastName = 'Johnson' AND FirstName = 'Emily';
```

-- Create Composite Index

```
CREATE NONCLUSTERED INDEX IX_Person_FullName
```

```
ON Person.Person (LastName, FirstName);
```

-- Query AFTER

```
SELECT BusinessEntityID
```

```
FROM Person.Person
```

```
WHERE LastName = 'Johnson' AND FirstName = 'Emily';
```

6 Covering Index (INCLUDE)

Scenario: Query CustomerID + OrderDate + TotalDue.

-- Query BEFORE

```
SELECT CustomerID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE CustomerID = 11000;
```

```
-- Create Covering Index
CREATE NONCLUSTERED INDEX IX_SalesOrder_Customer
ON Sales.SalesOrderHeader (CustomerID)
INCLUDE (OrderDate, TotalDue);
```

```
-- Query AFTER
SELECT CustomerID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE CustomerID = 11000;
👉 Execution Plan: Key Lookup eliminated.
```

7 Columnstore Index

Scenario: Reporting query on sales.

```
-- Create Clustered Columnstore
CREATE CLUSTERED COLUMNSTORE INDEX CCI_SalesOrderDetail
ON Sales.SalesOrderDetail;
```

```
-- Query AFTER
SELECT ProductID, SUM(LineTotal) AS TotalSales
FROM Sales.SalesOrderDetail
GROUP BY ProductID;
👉 Compare speed vs traditional rowstore.
```

8 XML Index

Scenario: Search inside XML column.

```
-- Query BEFORE
SELECT Name
FROM Production.ProductModel
WHERE CatalogDescription.value('(/ProductDescription/Summary)[1]', 'nvarchar(100)')
LIKE '%bike%';
```

```
-- Create Primary XML Index
CREATE PRIMARY XML INDEX PXML_ProductModel
ON Production.ProductModel (CatalogDescription);
```

```
-- Query AFTER
SELECT Name
FROM Production.ProductModel
WHERE CatalogDescription.value('(/ProductDescription/Summary)[1]', 'nvarchar(100)')
LIKE '%bike%';
```

9 Spatial Index

Scenario: Nearest stores.

-- Assume StoreLocations table with GeoLocation

```
CREATE TABLE dbo.StoreLocations
```

```
(
```

```
    StoreID INT PRIMARY KEY,
```

```
    StoreName NVARCHAR(100),
```

```
    GeoLocation GEOGRAPHY
```

```
);
```

-- Spatial Index

```
CREATE SPATIAL INDEX SInd_Store_Geo
```

```
ON dbo.StoreLocations (GeoLocation);
```

-- Query

```
SELECT StoreName
```

```
FROM dbo.StoreLocations
```

```
WHERE GeoLocation.STDistance(geography::Point(40.7128, -74.0060, 4326)) < 5000;
```

10 Full-Text Index

Scenario: Search documents.

-- Enable full-text

```
CREATE FULLTEXT CATALOG ftCatalog AS DEFAULT;
```

```
CREATE FULLTEXT INDEX ON Production.Document
```

```
(DocumentSummary LANGUAGE 1033)
```

```
KEY INDEX PK_Document_DocumentID;
```

-- Query

```
SELECT DocumentID, Title
```

```
FROM Production.Document
```

```
WHERE CONTAINS(DocumentSummary, '"safety helmet" OR "protective gear"');
```

Learnings Suggestion:

1. Run query without index (show scan).
2. Create index.
3. Rerun query (show seek or reduced cost).
4. Compare Execution Plans + I/O stats.
5. Discuss trade-offs (storage, maintenance, fragmentation).

Scripts ready-to-use .sql lab file (well-structured with comments) so you can directly import it into SSMS for practice:

-- AdventureWorks2019 Index Demo Lab Script
-- Author: Sage (SQL Server Training)
-- Purpose: Demonstrate different index types with before/after performance

-- Prep Settings

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

/*=====

1. Clustered Index Demo

=====*/

-- Before

SELECT SalesOrderID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE SalesOrderID = 50000;
GO

-- Create Clustered Index

CREATE CLUSTERED INDEX IX_SalesOrderHeader_SalesOrderID
ON Sales.SalesOrderHeader (SalesOrderID);
GO

-- After

SELECT SalesOrderID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE SalesOrderID = 50000;
GO

/*=====

2. Non-Clustered Index Demo

=====*/

-- Before

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE LastName = 'Smith';
GO

-- Create Non-Clustered Index

CREATE NONCLUSTERED INDEX IX_Person_LastName
ON Person.Person (LastName);
GO

-- After

```
SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE LastName = 'Smith';
GO
```

/*=====

3. Unique Index Demo

=====*/

```
CREATE UNIQUE INDEX IX_Email_Unique
ON Person.EmailAddress (EmailAddress);
GO
```

-- Test Duplicate Insert (Should Fail)

```
-- INSERT INTO Person.EmailAddress (BusinessEntityID, EmailAddress, rowguid, ModifiedDate)
-- VALUES (1, 'ken0@adventure-works.com', NEWID(), GETDATE());
GO
```

/*=====

4. Filtered Index Demo

=====*/

-- Before

```
SELECT SalesOrderID, OrderDate, Status
FROM Sales.SalesOrderHeader
WHERE Status = 1;
GO
```

-- Create Filtered Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrder_Pending
ON Sales.SalesOrderHeader (OrderDate, CustomerID)
WHERE Status = 1;
GO
```

-- After

```
SELECT SalesOrderID, OrderDate, Status
FROM Sales.SalesOrderHeader
WHERE Status = 1;
GO
```

/*=====

5. Composite Index Demo

=====*/

-- Before

```
SELECT BusinessEntityID
FROM Person.Person
WHERE LastName = 'Johnson' AND FirstName = 'Emily';
GO
```

-- Create Composite Index

```
CREATE NONCLUSTERED INDEX IX_Person_FullName
ON Person.Person (LastName, FirstName);
GO
```

-- After

```
SELECT BusinessEntityID
FROM Person.Person
WHERE LastName = 'Johnson' AND FirstName = 'Emily';
GO
```

```
/*=====
6. Covering Index Demo
=====*/
```

-- Before

```
SELECT CustomerID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE CustomerID = 11000;
GO
```

-- Create Covering Index

```
CREATE NONCLUSTERED INDEX IX_SalesOrder_Customer
ON Sales.SalesOrderHeader (CustomerID)
INCLUDE (OrderDate, TotalDue);
GO
```

-- After

```
SELECT CustomerID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE CustomerID = 11000;
GO
```

```
/*=====
7. Columnstore Index Demo
=====*/
```

-- Create Clustered Columnstore Index

```
CREATE CLUSTERED COLUMNSTORE INDEX CCI_SalesOrderDetail
ON Sales.SalesOrderDetail;
GO
```

-- Query

```
SELECT ProductID, SUM(LineTotal) AS TotalSales
FROM Sales.SalesOrderDetail
GROUP BY ProductID;
GO
```

```
/*=====
8. XML Index Demo
=====*/

-- Before
SELECT Name
FROM Production.ProductModel
WHERE CatalogDescription.value('(/ProductDescription/Summary)[1]', 'nvarchar(100)') LIKE '%bike%';
GO

-- Create Primary XML Index
CREATE PRIMARY XML INDEX PXML_ProductModel
ON Production.ProductModel (CatalogDescription);
GO

-- After
SELECT Name
FROM Production.ProductModel
WHERE CatalogDescription.value('(/ProductDescription/Summary)[1]', 'nvarchar(100)') LIKE '%bike%';
GO

/*=====
9. Spatial Index Demo (Custom Table)
=====*/
IF OBJECT_ID('dbo.StoreLocations') IS NOT NULL DROP TABLE dbo.StoreLocations;
GO

CREATE TABLE dbo.StoreLocations
(
    StoreID INT PRIMARY KEY,
    StoreName NVARCHAR(100),
    GeoLocation GEOGRAPHY
);
GO

-- Create Spatial Index
CREATE SPATIAL INDEX SInd_Store_Geo
ON dbo.StoreLocations (GeoLocation);
GO

-- Query
-- SELECT StoreName
-- FROM dbo.StoreLocations
-- WHERE GeoLocation.STDistance(geography::Point(40.7128, -74.0060, 4326)) < 5000;
GO
```

```
/*=====
10. Full-Text Index Demo
=====*/

-- Enable Full-Text
CREATE FULLTEXT CATALOG ftCatalog AS DEFAULT;
GO

CREATE FULLTEXT INDEX ON Production.Document
(DocumentSummary LANGUAGE 1033)
KEY INDEX PK_Document_DocumentID;
GO

-- Query
SELECT DocumentID, Title
FROM Production.Document
WHERE CONTAINS(DocumentSummary, '"safety helmet" OR "protective gear"');
GO
```

Here's a **visual flowchart for SQL Server index selection**  — showing the progression from **Clustered** → **Nonclustered** → **Filtered** → **Unique** → **Columnstore** → **Full-Text** → **XML/JSON**.

SQL Server Index Selection Flowchart

